

التمارين التي حلّها الطالب مسبقاً

بعد مراجعة محتوى المستودع والملفات، تبين أن الطالب قام بحل **عدة تمارين برمجية** من منهج المادة. أبرز التمارين التي وجد أنها محلولة في ملفاته تشمل ما يلي:

- **تمرين Queue (هيكل الطابور):** إنشاء صف `Queue` بخصائص إضافة عنصر وإرجاع العنصر التالي مع التحقق من الفراغ (يرفع استثناء عند كون الطابور فارغاً). هذا التمرين يتعلق بمفاهيم الإنكسوليشن (إخفاء البيانات) واستخدام البنى الداخلية (مثل قائمة `list` خاصة).
- **تمرين MusicalNote (نوتة موسيقية):** تعريف صف `MusicalNote` له خاصيتان للقراءة فقط هما `name` و `pitch` (الاسم والتردد). استخدم الطالب فيه أسلوب الخصائص (`@property`) لجعل المتغيرات ثابتة للقراءة فقط.
- **تمرين LengthConverter (محول الأطوال):** كتابة صف `LengthConverter` لتحويل القيم بين وحدات الطول (مثلاً من المتر إلى القدم). ورد في وصف التمرين أنه "تمرين أكثر تحدياً" يُختبر فهم الطالب للخصائص وإعادة الحساب ديناميكياً.
- **تمرين Money (المبالغ المالية وعملة النقد):** إنشاء صف `Money` يمثل مبلغاً معيناً بعملة محددة، مع تعريف عمليات الجمع والطرح (`__add__` و `__sub__`) بين كائنات المال مع التحقق من تطابق العملة. هذا التمرين اختبر قدرة الطالب على زيادة المشغل `operator overloading` في الأصفاف.
- **تمرين Human/Archer (توريث الكائنات):** سلسلة تمارين حول وراثة الأصفاف؛ بدأ الطالب بصنف `Human` (بخاصية الاسم) وصنف فرعي `Archer` يرث من `Human`. يتطلب التمرين تمرير الاسم للوالد ومعالجة سهم ال `Archer` الخاص به.
- **تمرين Crossbowman (رامي النبال):** تطوير إضافي على التمرين السابق بإضافة صف `Crossbowman` يرث من `Archer`. يميز الوصف أن `Crossbowman` هو دائماً `Archer` (رام)، لكن ليس كل رام هو `Crossbowman` مع خصائص إضافية مثل هجمة `triple_shot` وعدد أسهم مختلف. طلب التمرين أيضاً إضافة وظيفة `use_arrows` في صف `Archer` لمعالجة الأسهم والتحقق من عدم نفاذها.

هذه التمارين المذكورة أعلاه ظهرت في ملفات الطالب مع وجود حلول أو محاولات حل، مما يشير إلى أنه تدرب عليها مسبقاً.

التمارين الأقرب لمستوى تمارين الامتحان

بالنظر إلى وصف المستخدم لتمرين الامتحان على أنها **تمارين قوية ومركزة** تظهر مدى الفهم والسرعة والقدرة على كتابة كود واضح، يمكن تحديد التمارين التالية كعمالة في المستوى والأسلوب:

- **تطبيق Exercise Logger:** تمرين شامل (وارد ضمن عينات الامتحان) لتطوير تطبيق مصغر لتسجيل التمارين الرياضية. يحتوي على عدة أصفاف (`Run` , `Ride` , `Swim`) تشترك في خصائص (التاريخ، المسافة، المدة) وفي دوال مثل التحقق من صحة التاريخ واحتساب السرعات. يتطلب التمرين تصميم هيكل وراثي يجمع القواسم المشتركة بين هذه الأصفاف مع معالجة الاختلاف في طريقة حساب السرعات لكل نشاط. هذا التمرين مركّز ويختبر فهم الوراثة وتعدد الأشكال بشكل عملي.
- **تحليل بيانات الطقس (Weather Data Analysis):** تمرين عملي (وارد أيضاً ضمن عينات الامتحان) على **معالجة البيانات باستخدام مكتبة Pandas**. يُعطى ملف بيانات CSV يحتوي على معلومات الطقس لعدة مدن، ويُطلب من الطالب تنظيف البيانات وتحليلها للإجابة على أسئلة محددة. يتطلب ذلك سرعة في فهم البيانات واستخدام

دوال المكتبة بفعالية (مثل قراءة الملف، التلخيص من القيم الشاذة أو الناقصة، حساب إحصاءات مثل متوسطات الحرارة، ... إلخ). التركيز هنا على **الفهم العميق** لكيفية معالجة البيانات ووضوح الخطوات المتبعة برمجياً.

تمارين التعبيرات النمطية (Regex) المتقدمة: من الأمثلة على ذلك تحقق من البريد الإلكتروني الصحيح أو كلمة المرور الصحيحة. على سبيل المثال، تمرين "التحقق من صحة عنوان البريد الإلكتروني" يطلب كتابة دالة `is_valid_email_address` تتأكد أن سلسلة نصية معينة تمثل بريداً إلكترونياً صالحاً وفق معايير محددة. هذه التمارين مركزة جداً وتتطلب فهماً سريعاً لبناء التعبيرات النمطية واستعمالها بفعالية (مثل استخدام `^` و `$` والمجموعات الاختيارية والتكرار). وتظهر قدرة الطالب على كتابة كود صحيح وقوي من المحاولة الأولى أو بسرعة تصحيح الأخطاء.

باستخدام المعلومات أعلاه، فيما يلي **خطة مراجعة تفصيلية لمدة 5 أيام** (بمعدل ٣ ساعات يومياً) تراجع أهم المواضيع بشكل مركز وتتدرب على تمرين قوي كل يوم:

اليوم	المراجعة السريعة (Flashback)	التمرين التطبيقي (تدريب عملي)
اليوم 1	مفاهيم البرمجة كائنية التوجه (OOP) الأساسية: تعريف الأنصاف والكائنات، الفُنشئات (<code>__init__</code>)، المفاهيم الأساسية للإنكسوليشن وإخفاء البيانات (استخدام المتغيرات الخاصة <code>__var</code>)، خصائص الوصول (<code>@property</code> للكتابة فقط). أيضاً لمحة عن الطرق الساكنة <code>@staticmethod</code> وطرق الصف <code>@classmethod</code> عند الحاجة.	تمرين عملي: إعادة حل تمرين Queue البسيط. يشمل ذلك إنشاء صف <code>Queue</code> بواجهة أساسية: دالة <code>add(item)</code> لإضافة عنصر، دالة <code>next()</code> لإخراج أول عنصر في الصف (مع رفع خطأ من نوع <code>ValueError</code> إذا كان الصف فارغاً)، ودالة <code>is_empty()</code> للتحقق من خلو الصف. هذا التمرين يختبر استيعاب مبادئ OOP الأساسية ويعزز كتابة كود واضح (مثل استخدام أسماء معبرة والتعامل الصحيح مع الاستثناءات).
اليوم 2	الوراثة والتعددية Polymorphism في OOP: مراجعة سريعة لكيفية اشتقاق صف فرعي من صف أساسي (<code>class</code> <code>SubClass(BaseClass):</code> باستخدام الدالة <code>super().__init__()</code> لاستدعاء مُنشئ الوالد. مناقشة مفاهيم تجاوز الطرق (<code>method overriding</code>) وكيفية استخدام الأنصاف المجردة (مثال: جعل دالة <code>calories</code> مجردة يتعين على الأنصاف الفرعية تنفيذها). أيضاً التأكيد على أهمية تصميم واجهات واضحة للأنصاف بحيث يسهل استخدامها بسرعة.	تمرين عملي: تطوير تطبيق Exercise Logger المصغّر. قم بإنشاء صف عام <code>Exercise</code> يجمع الخصائص المشتركة (<code>distance</code>, <code>date</code>, <code>duration</code>) ودوال مثل <code>is_valid_date()</code> للتحقق من صحة التاريخ. ثم اشتق منه الأنصاف الفرعية <code>Run</code> و <code>Ride</code> و <code>Swim</code>، مع تنفيذ كل منها لطريقة <code>calories()</code> وفق صيغة مختلفة (مثلاً الجري يحرق سرعات أكثر من السباحة لكل كيلومتر وهكذا). تأكد من استدعاء مُنشئ الوالد في كل صف فرعي لوراثة الخصائص المشتركة. هذا التمرين يقارب مستوى تعقيد سؤال الامتحان، لذا حاول كتابته بوتيرة معتدلة ثم راجع الكود لتحسين الوضوح والبساطة - مثلاً استخدام أسماء واضحة للمتغيرات وتجنب التكرار بين الأنصاف عبر الوراثة.

اليوم
3

البرمجة الوظيفية والعودة للتعريفات الرياضية (Recursion & Functional Programming): لمحة سريعة حول المفاهيم الوظيفية في بايثون: توابع المرتفعة (higher-order functions) التي تأخذ أو ترجع توابع، التعبيرات lambda المختصرة، comprehension للقوائم والمجموعات... إلخ. ثم تركيز خاص على مفهوم الاستدعاء الذاتي (Recursion): كيفية تقسيم المشكلة لمشاكل أصغر، شرط التوقف، أمثلة بسيطة (مثل دالة تحسب المضروب أو مجموع قائمة أرقام باستخدام الاستدعاء الذاتي). ناقش أيضًا فوائده ومخاطر recursion (مثل إمكانية حدوث stack overflow إذا لم يُضبط شرط التوقف).

تمرين عملي: حل تمرين Fractals (الفركتلات - الأشكال الهندسية الذاتية التشابه). هذا التمرين متقدم ويتطلب تطبيق الاستدعاء الذاتي في رسم شكل شجري متفرع (حرف Y متكرر). ابدأ بوصف الفكرة: دالة ترسم فرعًا ثم تستدعي نفسها لرسم الفروع الفرعية وهكذا. يمكن الاستعانة بمكتبة مثل turtle أو matplotlib لرسم الشكل إذا كان ذلك جزءًا من التمرين. الهدف ليس إخراج رسم جميل فقط، بل فهم نمط التفكير في حل المشكلة بالاستدعاء الذاتي. بعد الانتهاء، راجع الحل وتأمل كيف ينطلق الاستدعاء الذاتي من الحالة الأساس Base Case باتجاه الحل الكامل. (إن لم يتسلسل الرسم فعليًا، يمكن pseudocode يطبع بنية الفركتل كنص لمتابعة منطق recursion).

اليوم
4

التعبيرات النمطية (Regular Expressions): واختبار البرمجيات: استعرض سريعًا بنية التعبيرات النمطية في بايثون (re module): الرموز الخاصة مثل { } [] * ? + . \$ ^ وكيف استخدمها لبناء نمط matching قوي. ذكّر ببعض الاستراتيجيات لبناء regex معقد: مثلًا البدء بنمط بسيط وتجربته ثم زيادة التعقيد تدريجيًا، واستخدام raw strings في بايثون مثل r'\d+'. كذلك تطرّق بإيجاز لفكرة اختبار الوحدة (Unit Testing) - كيف يمكن كتابة اختبارات سريعة أو استخدام حالات اختبار للتأكد من صحة التعبير النمطي أو الدوال المكتوبة (مفيد للتحقق من الحلول بسرعة أثناء الامتحان).

تمرين عملي: كتابة دالة للتحقق من صحة بريد إلكتروني باستخدام regex. طبق تمرين "التحقق من عنوان البريد الإلكتروني الصحيح": صمم التعبير النمطي بحيث يتحقق من وجود جزء اسم المستخدم (حروف وأرقام ونقاط مسموحة مثلًا)، متبوعًا بعلامة @، ثم نطاق domain صالح (حروف أو أرقام، ونقطة، ثم امتداد). تأكد من تغطية بعض الحالات الخاصة (مثل عدم السماح ببداية بـ .) أو وجود رموز غير صالحة). يمكنك اختبار الدالة على أمثلة معروفة: مثلًا test@example.com يجب أن يعطي True، و invalid@test يجب أن يعطي False، وهكذا. التركيز هنا على كتابة Regex واضح وصحيح بسرعة نسبيًا؛ إذا واجهت صعوبة، قسم المشكلة (مثل التحقق من كل قسم من البريد على حدة) ثم جمع الحل. (ملاحظة: يمكن أيضًا تطبيق نفس الفكرة على تمرين التحقق من كلمة المرور القوية إن أردت مزيدًا من التحدي).

تمرين عملي: تحليل بيانات الطقس (Weather)

(Analysis) - نفذ خطوات عملية على مجموعة بيانات الطقس المشابهة لتمرين الامتحان. افتح ملف `weather_dataset.csv` (افتراضياً) وابدأ بتنظيف البيانات: تأكد من إزالة أو تعبئة القيم الناقصة، وتوحيد الوحدات إذا لزم الأمر. ثم أجب عن أسئلة تحليلية كما يُطلب عادة: مثلاً **ما المدينة ذات أعلى درجة حرارة مسجلة؟** وما متوسط سرعة الرياح لكل مدينة؟ (افتراضات للأسئلة الممكنة). قم بتجميع البيانات حسب المدينة أو التاريخ باستخدام `groupby` لحساب المتوسطات أو القيم القصوى. أخيراً، يمكنك إنشاء **رسم بياني بسيط** يوضح معلومة مستخلصة (مثلاً مخطط للأعلى حرارة في كل مدينة). أثناء الحل، حافظ على كود واضح ومعلق عليه بشكل مختصر إذا أمكن، ليكون من السهل فهم خطواتك. هذا التمرين الشامل يختبر قدرتك على توظيف ما تعلمته في **وقت محدود** وبطريقة منظمة للحصول على نتائج صحيحة.

معالجة البيانات والتحليل باستخدام

المكتبات: راجع بسرعة طريقة استخدام **مكتبة**

Pandas في بايثون: قراءة الملفات

(`pd.read_csv`)، عرض `DataFrame` والتعرف

على هيكله، عمليات التنظيف الأساسية (حذف

الصفوف أو القيم الناقصة `dropna`، التعامل

مع القيم الشاذة)، عمليات التحليل مثل التجميع

`groupby` والحصول على ملخصات إحصائية

(`mean()`, `max()`, `min()`) وغيرها). كذلك

لمحة عن رسم البيانات إن كان مشمولاً

بإستخدام `matplotlib` أو

`pandas.plot`). ذكر نفسك أيضاً بكيفية

كتابة الكود في Jupyter Notebook بسرعة

وترتيب لإظهار النتائج خطوة بخطوة، لأن

الامتحان قد يكون في بيئة مشابهة.

ملاحظات ختامية: تأكد خلال المراجعة اليومية من الاستفادة من نمط الشرح والكود المستخدم في المادة الدراسية نفسها كمصدر أساسي - راجع حلولك السابقة ونصائح الأساتذة في المحاضرات/المختبر. في كل يوم، ابدأ بـ فلاش باك سريع للمفاهيم لضمان أنك تتذكر الأساسيات، ثم انتقل للتطبيق العملي. حاول ضبط وقت لكل جزء من التمرين اليومي لتعويد نفسك على الحل السريع تحت الضغط. وأخيراً، لا بأس باستخدام أمثلة خارجية إضافية للتعلم بشرط الحفاظ على نفس أسلوب وصياغة الكود المتبع في الكورس لضمان الألفة والوضوح. بهذه الخطة المتدرجة ستغطي أهم جوانب المادة وتستعيد ثقتك بسرعتك في الحل ووضوحك في كتابة الشفرة، استعداداً ليوم الامتحان بإذن الله.